



Universidad Autónoma del Estado de México



Facultad de Ingeniería

Evidencias(pantallas) Proyecto(simulador)

RAMIRO VEGA MEZA

rvegam003@alumno.uaemex.mx

Licenciatura:

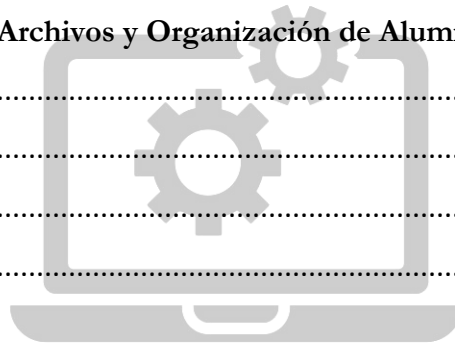
INGENIERIA EN COMPUTACION

SISTEMAS OPERATIVOS

Toluca, México

Contenido

P1 Simulador de Sistema de Archivos y Organización de Alumnos.....	3
FCFS (First-Come, First-Served)	4
SJF (Shortest Job First)	5
SRTF (Shortest Remaining Time First)	6
RR (Round Robin)	7
P2 Algoritmos de Reemplazo de Página	8
FIFO (First-In, First-Out)	9
LRU (Least Recently Used)	10
Óptimo (Optimal Page Replacement).....	11
Clock (Algoritmo del Reloj).....	12
Second Chance	13
P3 Simulador de Sistema de Archivos y Organización de Alumnos	15
Pile	21
Secuencial	22
Secuencial Indexado	23
Hash	24



P1 Simulador de Sistema de Archivos y Organización de Alumnos

Este módulo permite simular diferentes algoritmos de planificación de CPU (RR, FCFS, SJF, SRTF) para comprender cómo se asigna la CPU a los procesos y calcular métricas como tiempo de espera, respuesta y turnaround.

Esta sección permite simular cómo el sistema operativo planifica procesos en la CPU.

Instrucciones:

1. Selecciona el algoritmo en el combo:

- RR (Round Robin)
- FCFS (First Come First Served)
- SJF (Shortest Job First)
- SRTF (Shortest Remaining Time First)

2. Selecciona el algoritmo en el combo:

- RR (Round Robin)
- FCFS (First Come First Served)
- SJF (Shortest Job First)
- SRTF (Shortest Remaining Time First)

3. Ingresar los datos del proceso:

- Tiempo de llegada (≥ 0)
- Ráfaga (Burst Time) (> 0)
- Si el algoritmo es RR, también ingresa el Quantum (> 0).

4. Agrega el proceso:

- Presiona “Agregar Proceso”.
- El proceso aparecerá en la tabla con ID automático (P1, P2, ...).

5. Eliminar proceso (opcional):

- Presiona “Eliminar” para quitar el último proceso agregado.

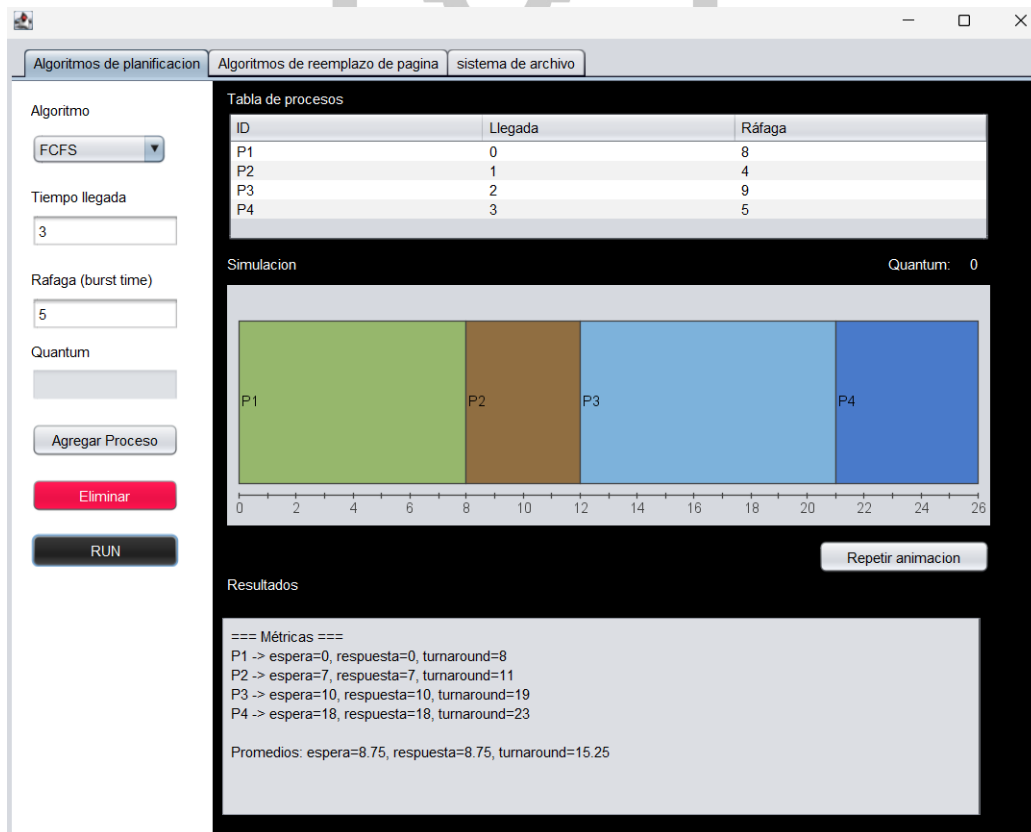
6. Ejecutar simulación:

- Presiona “RUN”.
- Se mostrará:
 - Diagrama de Gantt con la ejecución.
 - Métricas por proceso: tiempo de espera, respuesta y turnaround.
 - Promedios de cada métrica.

FCFS (First-Come, First-Served)

FCFS atiende los procesos en el orden en que llegan. No hay interrupciones ni prioridades:

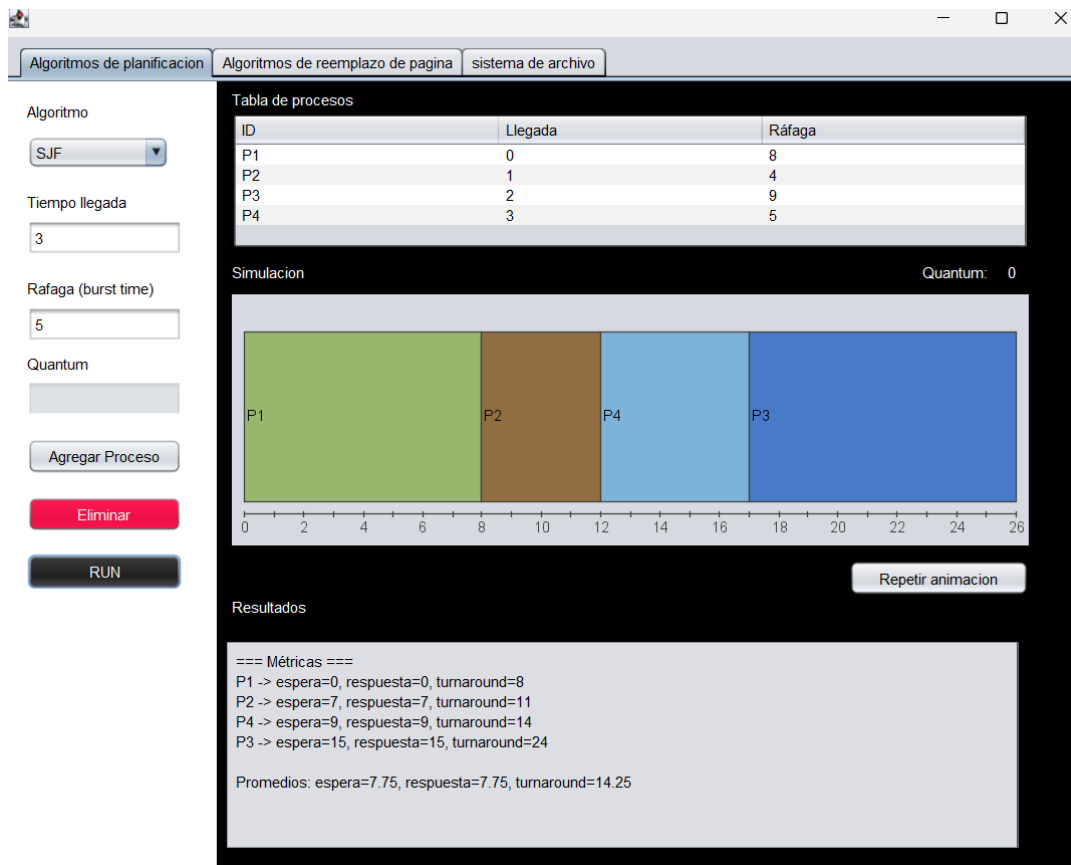
- El primer proceso que llega se ejecuta hasta terminar.
- Los demás esperan en la cola.
- Es simple, pero puede generar efecto convoy (procesos cortos esperan mucho si hay uno largo primero).



SJF (Shortest Job First)

SJF selecciona el proceso con la ráfaga más corta entre los que están listos para ejecutar. Es no apropiativo:

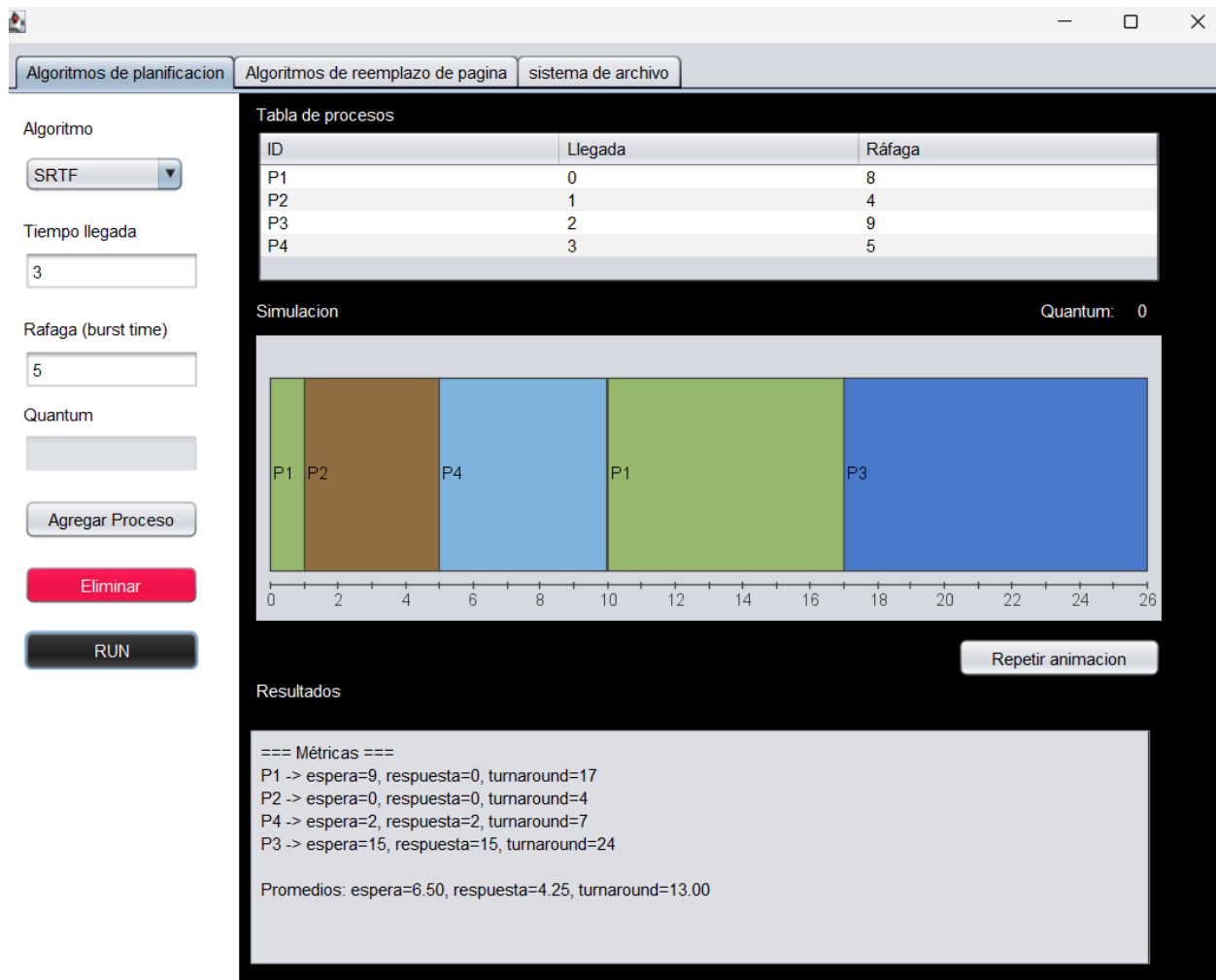
- Se ejecuta el proceso más corto disponible hasta terminar.
- Reduce el tiempo promedio de espera comparado con FCFS.
- Puede causar injusticia: procesos largos esperan mucho si llegan después.



SRTF (Shortest Remaining Time First)

SRTF es la versión apropiativa de SJF. El proceso con el menor tiempo restante se ejecuta primero, y puede ser interrumpido si llega otro más corto:

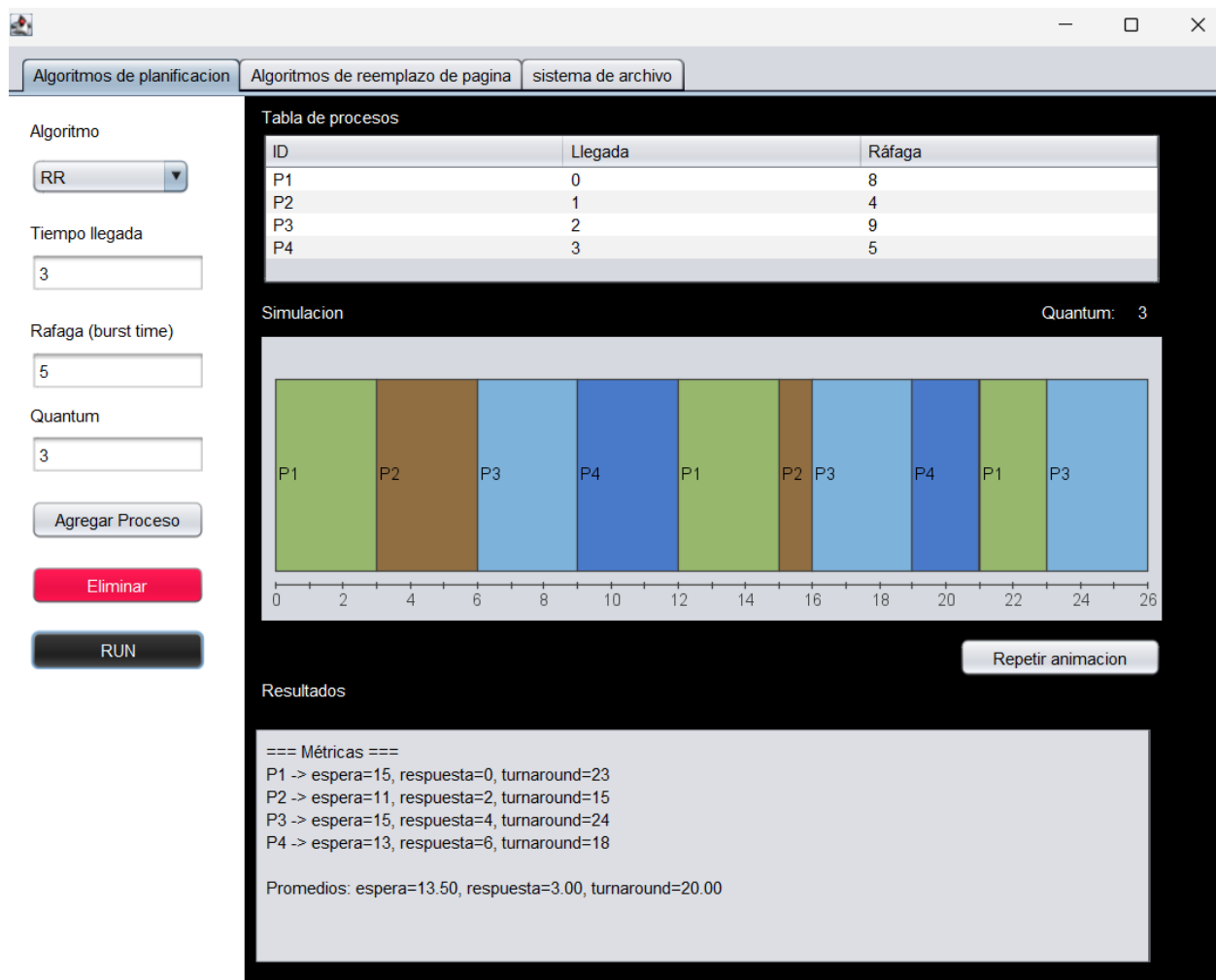
- Si un proceso nuevo tiene menor ráfaga restante, se interrumpe el actual.
- Reduce el tiempo promedio de espera más que SJF.
- Puede generar muchas interrupciones y sobrecarga por cambio de contexto.



RR (Round Robin)

RR asigna la CPU a cada proceso por un quantum fijo en forma circular. Si el proceso no termina en su turno, vuelve al final de la cola:

- Cada proceso recibe un tiempo limitado (quantum) para ejecutarse.
- Mejora la justicia entre procesos, evitando que uno monopolice la CPU.
- Puede aumentar el tiempo total de turnaround si el quantum es muy pequeño (muchos cambios de contexto).

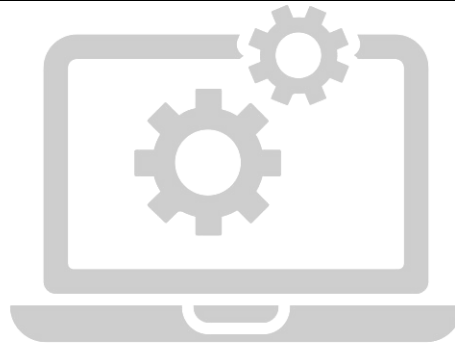


P2 Algoritmos de Reemplazo de Página

Este módulo permite simular cómo los sistemas operativos manejan la memoria virtual mediante algoritmos de reemplazo de página (FIFO, LRU, Óptimo, Clock, Second Chance, Working Set, VMIN, PFF), mostrando fallos de página, hits y tasa de fallos.

Instrucciones:

1. Selecciona el algoritmo.
2. Agrega páginas (A, B, C, D, E) usando Agregar Pagina.
3. Presiona RUN, ingresa el número de marcos y parámetros adicionales si el algoritmo lo requiere.
4. Observa: Panel gráfico con la simulación. Resumen con fallos, hits y tasa de fallo



FIFO (First-In, First-Out)

FIFO reemplaza la página más antigua cuando ocurre un fallo y no hay marcos libres:

- Las páginas se cargan en el orden en que llegan.
- Si la memoria está llena, se expulsa la que lleva más tiempo en memoria.
- Es simple, pero puede provocar **anomalía de Belady** (más marcos no siempre significa menos fallos).

The screenshot shows a web-based simulation interface for the FIFO page replacement algorithm. The interface is divided into several sections:

- Algoritmos de planificación**: A tabbed menu with 'Algoritmos de planificación', 'Algoritmos de reemplazo de pagina', and 'sistema de archivo'. The 'Algoritmos de reemplazo de pagina' tab is selected.
- Algoritmo**: A dropdown menu set to 'FIFO'.
- Paginas**: A dropdown menu set to 'D'.
- Buttons**: 'Agregar Pagina' (pink) and 'RUN' (black).
- Secuencia de Paguinas**: A list of pages: A, B, C, D.
- Simulacion**: A section showing the state of memory frames at different steps.
 - Estado inicial**: Three frames, all labeled 'null'. Below it, the text 'Estado inicial: todos los marcos vacíos.'.
 - Paso 1: ENTRA A (Falla)**: The first frame now contains 'A', while the other two are 'null'. Below it, the text 'Falla: reemplaza en marco 0 por A.'.
 - Paso 2: ENTRA B (Falla)**: The first frame contains 'A' and the second frame contains 'B', while the third is 'null'. Below it, the text 'Falla: reemplaza en marco 1 por B.'.
- Resultados**: A text area displaying the simulation results:

```
Algoritmo: FIFO
Secuencia: A, B, C, D, A, B, E, C, D
Marcos: 3

Resultado final: E | C | D
Fallas: 9
Hits: 0
Tasa de fallas: 100.00%
```
- Footer**: A blue information icon and a 'Limpiar Todo' button.

LRU (Least Recently Used)

LRU reemplaza la página que **no se ha usado por más tiempo** cuando ocurre un fallo:

- Mantiene un historial del último uso de cada página.
- Si hay espacio, coloca la página; si no, expulsa la menos recientemente usada.
- Es más eficiente que FIFO en muchos casos, pero requiere **más control y memoria** para rastrear el uso.

The screenshot shows a simulation window for the LRU algorithm. On the left, there are controls for the algorithm (set to LRU) and the sequence of pages (set to A). The main area displays the sequence of pages (A, B, C, D) and the state of three memory frames. The initial state shows all frames as null. Step 1 shows page A entering frame 0, causing a fault. Step 2 shows page B entering frame 1, causing a fault. The results section at the bottom shows the sequence of pages, the number of frames (3), the final state of frames (A | C | D), the number of faults (10), hits (0), and the fault rate (100.00%).

Algoritmos de planificacion | Algoritmos de reemplazo de pagina | sistema de archivo

Algoritmo: LRU

Paginas: A

Agregar Pagina

RUN

Secuencia de Paginas

Pagina
A
B
C
D

Simulacion

Estado inicial: null null null

Estado inicial: todos los marcos vacios.

Paso 1: ENTRA A (Falla)

A	null	null
---	------	------

Falla: reemplaza en marco 0 por A.

Paso 2: ENTRA B (Falla)

A	B	null
---	---	------

Falla: reemplaza en marco 1 por B.

Resultados

```
Algoritmo: LRU
Secuencia: A, B, C, D, A, B, E, C, D, A
Marcos: 3

Resultado final: A | C | D
Fallas: 10
Hits: 0
Tasa de fallas: 100.00%
```

Limpiar Todo

Óptimo (Optimal Page Replacement)

El algoritmo Óptimo reemplaza la página que **no se usará por más tiempo en el futuro**:

- Minimiza el número de fallos porque siempre elige la víctima ideal.
- Es **teórico**, ya que requiere conocer las referencias futuras (no se aplica en sistemas reales).
- Sirve como referencia para comparar otros algoritmos.

The application interface is divided into three main sections:

- Left Sidebar:**
 - Algoritmos de planificación: Algoritmos de reemplazo de página (selected), sistema de archivo
 - Algoritmo: Optimal Page Replacement (dropdown)
 - Páginas: A (dropdown)
 - Buttons: Agregar Página (pink), RUN (black)
 - Buttons: i (info), Limpiar Todo (clear)
- Main Simulation Area:**
 - Secuencia de Páginas:** A, B, C, D
 - Simulación:**
 - Estado inicial: null, null, null. Estado inicial: todos los marcos vacíos.
 - Paso 1: ENTRA A (Falla). Frame 0 contains A. Falla: reemplaza en marco 0 por A.
 - Paso 2: ENTRA B (Falla). Frame 1 contains B. Falla: reemplaza en marco 1 por B.
- Resultados:**

```
Algoritmo: Óptimo
Secuencia: A, B, C, D, A, B, E, C, D, A
Marcos: 3

Resultado final: A | C | D
Fallas: 6
Hits: 4
Tasa de fallas: 60.00%
```

Clock (Algoritmo del Reloj)

Clock es una variante del algoritmo de segunda oportunidad que utiliza un **puntero circular** y un **bit de uso** para decidir qué página reemplazar:

- Cada marco tiene un bit de referencia (u).
- Si hay espacio, coloca la página y marca $u=1$.
- Si no hay espacio:
 - El puntero recorre los marcos:
 - Si $u=0 \rightarrow$ reemplaza ahí.
 - Si $u=1 \rightarrow$ pone $u=0$ y avanza el puntero.
- Es eficiente y evita recorrer toda la lista como en Second Chance.

The screenshot shows a web-based simulation of the Clock algorithm. The interface includes a sidebar on the left with tabs for 'Algoritmos de planificacion', 'Algoritmos de reemplazo de pagina', and 'sistema de archivo'. The 'Algoritmos de reemplazo de pagina' tab is active, showing a dropdown for 'Algoritmo' set to 'Clock' and a dropdown for 'Paginas' set to 'A'. Below these are buttons for 'Agregar Pagina' and 'RUN'. The main area is titled 'Simulacion' and displays the 'Secuencia de Paginas' as A, B, C, D. It shows the initial state with three empty frames (null, u=0). Step 1: 'ENTRA A (Falla)' shows page A being placed in the first frame (u=1). Step 2: 'ENTRA B (Falla)' shows page B being placed in the second frame (u=1). The 'Resultados' section at the bottom provides a summary: 'Algoritmo: Clock', 'Secuencia: A, B, C, D, A, B, E, C, D, A, A', 'Marcos: 3', 'Resultado final: A | C | D', 'Fallas: 10', 'Hits: 1', and 'Tasa de fallas: 90.91%'. A 'Limpiar Todo' button is located at the bottom left.

Algoritmos de planificacion | Algoritmos de reemplazo de pagina | sistema de archivo

Algoritmo: Clock

Paginas: A

Agregar Pagina

RUN

Secuencia de Paginas

Pagina
A
B
C
D

Simulacion

Estado inicial:

null u=0	null u=0	null u=0
-------------	-------------	-------------

Estado inicial: todos los marcos vacios, puntero en posición 0.

Paso 1: ENTRA A (Falla)

A u=1	null u=0	null u=0
----------	-------------	-------------

Reemplaza en marco 0 por A.
Próximo puntero: posición 0.

Paso 2: ENTRA B (Falla)

A u=1	B u=1	null u=0
----------	----------	-------------

Reemplaza en marco 1 por B.

Resultados

```
Algoritmo: Clock
Secuencia: A, B, C, D, A, B, E, C, D, A, A
Marcos: 3

Resultado final: A | C | D
Fallas: 10
Hits: 1
Tasa de fallas: 90.91%
```

Limpiar Todo

Second Chance

Second Chance es una mejora del algoritmo FIFO que utiliza un **bit de referencia** para dar una “segunda oportunidad” a las páginas antes de reemplazarlas:

- Si hay espacio, coloca la página y marca $u=1$.
- Si no hay espacio:
 - Toma la página más antigua:
 - Si $u=0 \rightarrow$ reemplaza.
 - Si $u=1 \rightarrow$ pone $u=0$ y la reencola al final (segunda oportunidad).
- Reduce el riesgo de expulsar páginas que se usan con frecuencia.

The screenshot shows a web-based simulation interface for the Second Chance page replacement algorithm. The interface is divided into several sections:

- Algoritmos de planificacion**: Includes tabs for "Algoritmos de planificacion", "Algoritmos de reemplazo de pagina", and "sistema de archivo".
- Algoritmo**: A dropdown menu set to "Second Chance".
- Paginas**: A dropdown menu set to "A".
- Buttons**: "Agregar Pagina" (pink) and "RUN" (black).
- Secuencia de Paginas**: A list of pages: A, B, C, D.
- Simulacion**: The main simulation area showing the state of three frames (marcos) at different steps.
 - Estado inicial**: Three frames, each containing "null" with $u=0$ below it. Text below: "Estado inicial: todos los marcos vacíos."
 - Paso 1: ENTRA A (Falla)**: Frame 0 now contains "A" with $u=1$. Text below: "Reemplaza en marco 0 por A."
 - Paso 2: ENTRA B (Falla)**: Frame 1 now contains "B" with $u=1$. Text below: "Reemplaza en marco 1 por B."
- Resultados**: A text box showing the final results:

```
Algoritmo: Second Chance
Secuencia: A, B, C, D, A, B, E, C, D, A, A
Marcos: 3

Resultado final: A | C | D
Fallas: 10
Hits: 1
Tasa de fallas: 90.91%
```

Patrones que determinan coincidencias

1. FIFO, LRU y Óptimo pueden dar el mismo resultado si la secuencia no tiene repeticiones

- **Por qué:**
 - Si cada página aparece **solo una vez**, no hay oportunidad de hits.
 - Todos los algoritmos reemplazan páginas en el mismo orden porque:
 - FIFO: saca la más antigua.
 - LRU: todas son igualmente "no usadas recientemente" (porque no se repiten).
 - Óptimo: todas son igualmente "no usadas en el futuro" (porque no vuelven).
 - Resultado: **mismos fallos, mismos marcos finales.**
- **Ejemplo:**
Secuencia A, B, C, D con 3 marcos → todos terminan con [B, C, D] .

2. Cuando hay pocas repeticiones

- Secuencia: A, B, C, D, B, E
- Solo **B** se repite → hay **1 hit**.
- **Fallos:** iguales para todos (5 fallos, 1 hit).
- **Resultados finales:**
 - FIFO: D | E | C (porque sigue orden de llegada).
 - LRU: D | B | E (conserva B porque se usó recientemente).
 - Óptimo: E | B | C (elige víctimas según futuro).
 - Clock y Second Chance: D | B | E (similar a LRU en este caso).
- **Por qué difieren:**
 - El criterio de reemplazo cambia cuando hay repeticiones y empates.
 - Óptimo mira el futuro, LRU mira el pasado, FIFO ignora ambos.
- Óptimo mira el futuro, LRU mira el pasado, FIFO ignora ambos.

3. Clock y Second Chance siempre coinciden

- **Por qué:**
 - Ambos usan el **bit de uso** y un puntero circular.
 - La diferencia es solo en implementación (Clock usa puntero, Second Chance usa cola), pero la política es la misma.
- **Resultado:** siempre igual en marcos finales y fallos.

4. Casos donde todos coinciden

- Secuencia sin repeticiones y marcos < páginas únicas.
- Ejemplo: A, B, C, D, E con 3 marcos → todos terminan con [C, D, E] .

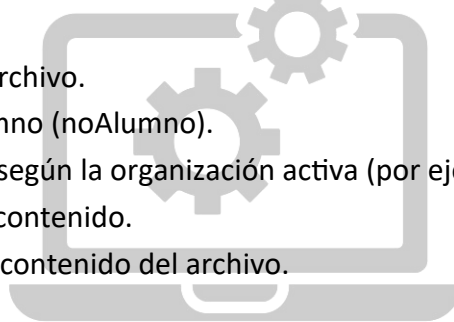
P3 Simulador de Sistema de Archivos y Organización de Alumnos

Objetivo del módulo

Este módulo permite simular un sistema de archivos con directorios y archivos, insertar registros de alumnos en diferentes organizaciones lógicas (Pile, Secuencial, Secuencial Indexado, Indexado, Hash), y visualizar tanto los archivos como el orden lógico según la organización activa.

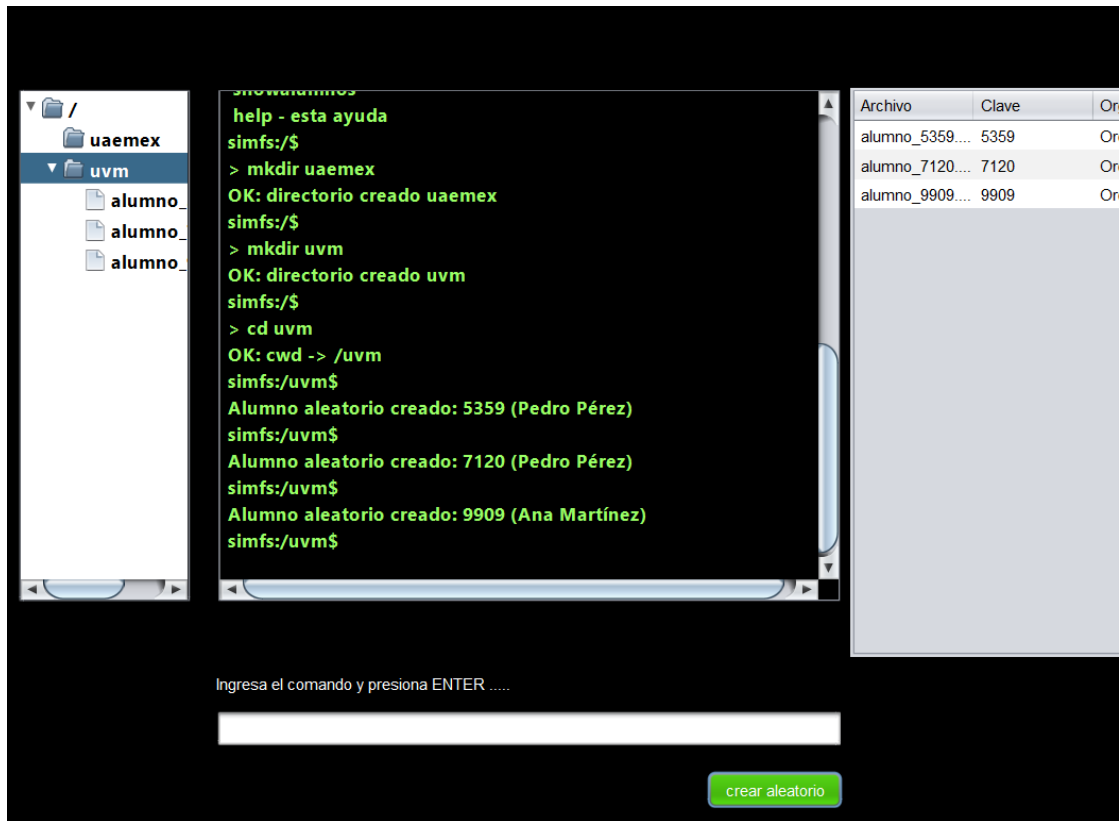
Componentes principales de la interfaz

- **Árbol de directorios (JTree):** muestra la estructura del sistema de archivos con íconos de carpeta y archivo.
- **Consola de comandos:** permite ejecutar instrucciones para manipular el sistema de archivos y la organización de alumnos.
- **Entrada de comandos:** campo donde se escribe el comando y se presiona ENTER.
- **Botón “Crear Aleatorio”:** genera un alumno con datos aleatorios, lo inserta en la organización activa y crea su archivo en el directorio actual.
- **Panel “Archivos (memoria)”:** muestra una tabla con los archivos del directorio actual y su información:
 - **Archivo:** nombre del archivo.
 - **Clave:** número de alumno (noAlumno).
 - **Org Info:** información según la organización activa (por ejemplo, posición, bucket, tipo).
 - **Tamaño:** longitud del contenido.
 - **Preview:** resumen del contenido del archivo.



Flujo de uso paso a paso

- Navegación en el sistema de archivos



Al iniciar la aplicación, aparece el mensaje:

Bienvenido. Escribe 'help'.

y el prompt:

simfs:/uvm\$

Para ver la lista de comandos disponibles, escribe:

help

Para crear un directorio:

mkdir uvm

Para entrar al directorio:

cd uvm

Para volver al directorio anterior:

cd ..

Para ir a la raíz:

cd /

Elegir organización lógica

Selecciona la organización para almacenar alumnos:

```
setorg pile
```

```
setorg sec
```

```
setorg sec_index
```

```
setorg indexado
```

```
setorg hash
```

pile: orden por inserción.

sec: orden por clave.

sec_index: orden + índice.

indexado: acceso directo por clave.

hash: distribución por buckets.

Nota: Al cambiar de organización, el sistema reconstruye la organización desde los archivos del directorio actual para no perder datos.

Organización de alumnos:

```
setorg <pile|sec|sec_index|indexado|hash>
```

```
insertalumno <no> "nombre" "apPat" "apMat" "tel" "
```

```
findalumno <no>
```

```
showalumnos
```

```
help - esta ayuda
```

```
simfs:/$
```

Insertar alumnos

Desde la consola:

```
insertalumno 8470 "Luis" "Pérez" "Ramírez" "5512345678" "Calle 5" "01010"
```

Esto:

Inserta el registro en la organización activa.

Crea el archivo alumno_8470.txt en el directorio actual con el contenido:

```
no=8470
nombre=Luis
apellidoPaterno=Pérez
apellidoMaterno=Ramírez
telefono=5512345678
calle=Calle 5
codigoPostal=01010
```

También puedes usar el botón **“Crear Aleatorio”**, que genera datos aleatorios, inserta el alumno y crea el archivo automáticamente.

4.4. Visualización en el panel

El panel muestra los archivos del directorio actual.

La columna **Org Info** refleja la organización activa:

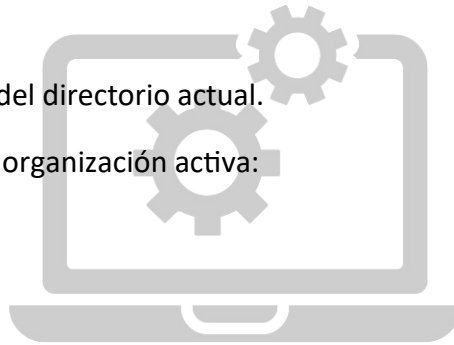
Pile (inserción)

Secuencial (orden clave)

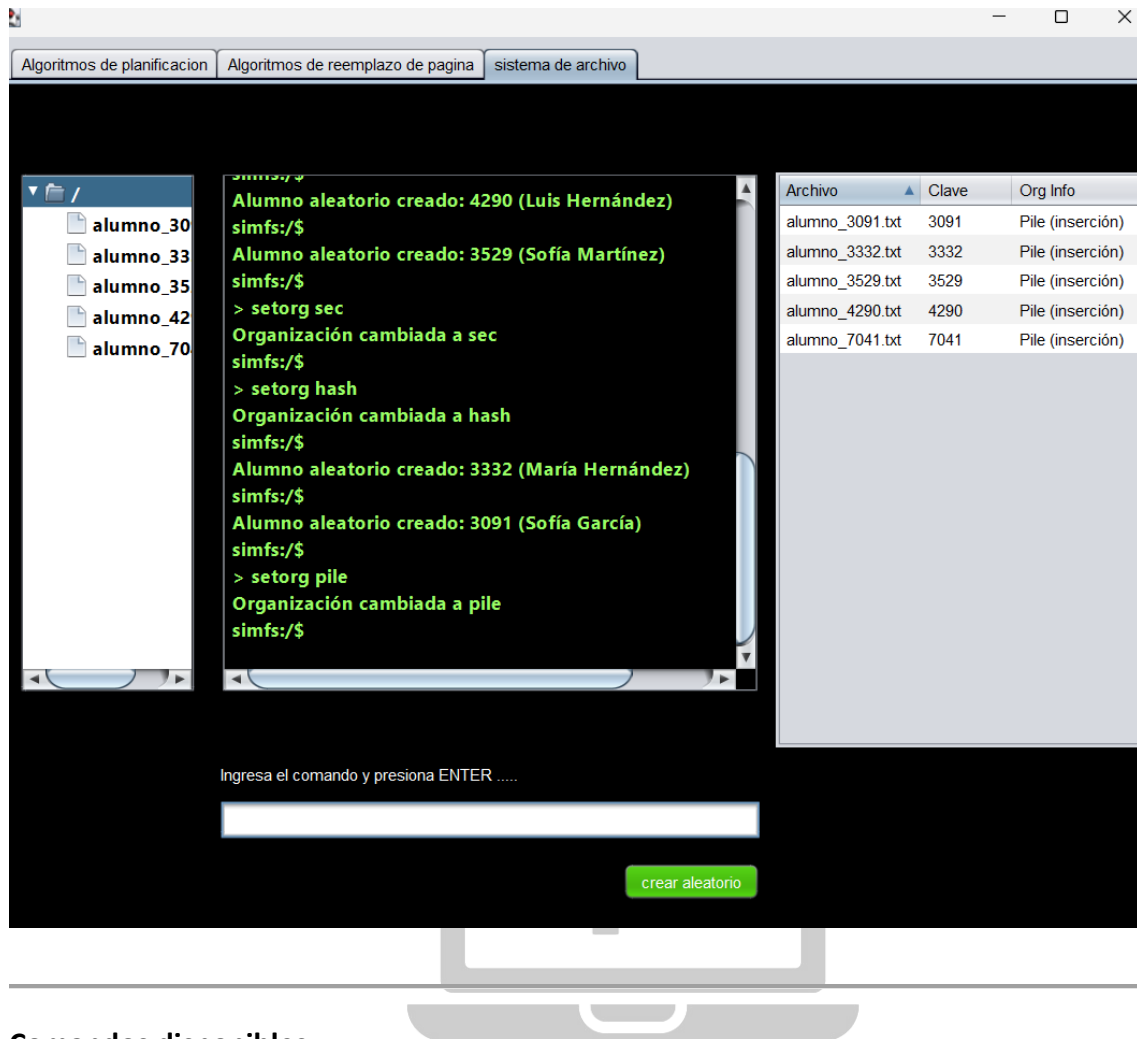
SecIndex pos=<n>

Indexado (map)

Hash bkt=<n>



Si cambias la organización con setorg, el panel se actualiza automáticamente.



Comandos disponibles

- **Comandos del sistema de archivos**

`pwd` → muestra el directorio actual.

`ls` → lista directorios y archivos.

`mkdir <nombre>` → crea un directorio.

`cd <ruta>` → cambia de directorio.

`touch <nombre>` → crea un archivo vacío.

`echo "texto" > archivo` → escribe contenido en un archivo.

- `echo "texto" >> archivo` → agrega contenido al final.
- `cat <archivo>` → muestra el contenido del archivo.

- `rm <archivo|dir>` → elimina archivo o directorio vacío.

Comandos de organización

- `setorg <pile|sec|sec_index|indexado|hash>` → cambia la organización activa.
- `insertalumno <no> "nombre" "apPat" "apMat" "tel" "calle" "cp"` → inserta un alumno.
- `findalumno <no>` → busca un alumno por clave.
- `showalumnos` → muestra la estructura interna de la organización.

6. Buenas prácticas

Antes de insertar alumnos, asegúrate de estar en el directorio correcto.

Si cambias la organización, el sistema reconstruirá la organización con los archivos del directorio actual.

Si el panel muestra “(sin orden org)”, significa que no hay datos en la organización activa (puede que estés en un directorio vacío).

Ejemplo completo

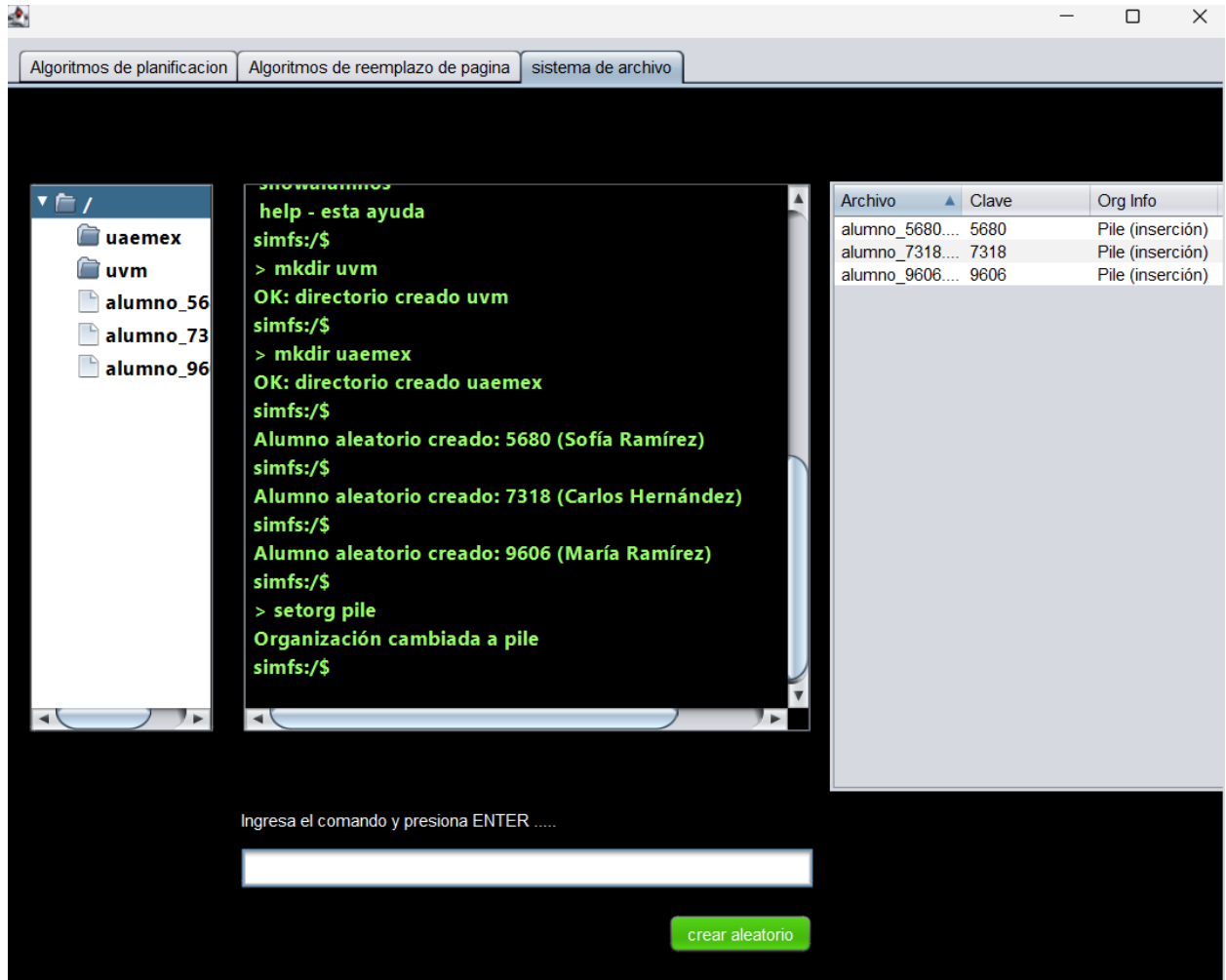
```
cd /
mkdir uvm
cd uvm
setorg hash
insertalumno 1001 "Ana" "García" "López" "5512340001" "Calle 1" "01010"
insertalumno 1002 "Luis" "Pérez" "Ramírez" "5512340002" "Calle 2" "02020"
```

Resultado en el panel:

Archivo	Clave	Org Info	Tamaño	Preview
alumno_1001.txt	1001	Hash bkt=XX	120	no=1001 nombre=Ana ...
alumno_1002.txt	1002	Hash bkt=YY	120	no=1002 nombre=Luis ...

Pile

- Inserta registros al final, sin orden.
- Si la clave existe, reemplaza el primero que coincida.
- Ventajas: simple y rápida para insertar.
- Desventajas: búsqueda lenta (lineal), no mantiene orden lógico.



Secuencial

- Inserta registros manteniendo orden por clave.
- Si la clave existe, reemplaza el registro.
- Ventajas: búsqueda más eficiente (puede ser binaria).
- Desventajas: inserción costosa (requiere reordenar).

The screenshot shows a software interface for simulating a file system. It has three tabs at the top: "Algoritmos de planificacion", "Algoritmos de reemplazo de pagina", and "sistema de archivo".

On the left is a file explorer showing a directory structure:

- /
- uaemex
- uvm
- alumno_56
- alumno_73
- alumno_96

The main area is a terminal window with the following text:

```
> mkdir uvm
OK: directorio creado uvm
simfs:/$
> mkdir uaemex
OK: directorio creado uaemex
simfs:/$
Alumno aleatorio creado: 5680 (Sofía Ramírez)
simfs:/$
Alumno aleatorio creado: 7318 (Carlos Hernández)
simfs:/$
Alumno aleatorio creado: 9606 (María Ramírez)
simfs:/$
> setorg pile
Organización cambiada a pile
simfs:/$
> setorg sec
Organización cambiada a sec
simfs:/$
```

On the right is a table with the following data:

Archivo	Clave	Org Info
alumno_5680....	5680	Secuencial (o...
alumno_7318....	7318	Secuencial (o...
alumno_9606....	9606	Secuencial (o...

At the bottom, there is a prompt "Ingresa el comando y presiona ENTER" and a text input field. Below the input field is a green button labeled "crear aleatorio".

Secuencial Indexado

- Mantiene registros ordenados y un índice primario (clave → posición).
- Inserción actualiza índice.
- Ventajas: búsqueda rápida por índice.
- Desventajas: mayor complejidad y uso de memoria.

The screenshot shows a file system simulation interface with three tabs: "Algoritmos de planificación", "Algoritmos de reemplazo de pagina", and "sistema de archivo". The "sistema de archivo" tab is active.

On the left, a file explorer shows a directory structure with folders "uaemex" and "uvm", and a series of files named "alumno_0" through "alumno_15".

The main terminal window displays the following commands and output:

```
Alumno aleatorio creado: 9997 (Carlos Hernández)
simfs:/$
> setorg indexado
Organización cambiada a indexado
simfs:/$
Alumno aleatorio creado: 2208 (María García)
simfs:/$
Alumno aleatorio creado: 2211 (Carlos Ramírez)
simfs:/$
Alumno aleatorio creado: 3981 (Juan Martínez)
simfs:/$
Alumno aleatorio creado: 6144 (Ana Pérez)
simfs:/$
Alumno aleatorio creado: 2304 (María García)
simfs:/$
> setorg sec_index
Organización cambiada a sec_index
simfs:/$
```

On the right, a table displays the file system state:

Archivo	Clave	Org Info
alumno_0553....	0553	SecIndex pos...
alumno_2208....	2208	SecIndex pos...
alumno_2211....	2211	SecIndex pos...
alumno_2304....	2304	SecIndex pos...
alumno_2770....	2770	SecIndex pos...
alumno_3981....	3981	SecIndex pos...
alumno_5680....	5680	SecIndex pos...
alumno_6144....	6144	SecIndex pos...
alumno_6196....	6196	SecIndex pos...
alumno_7318....	7318	SecIndex pos...
alumno_7632....	7632	SecIndex pos...
alumno_7918....	7918	SecIndex pos...
alumno_9606....	9606	SecIndex pos...
alumno_9997....	9997	SecIndex pos...

At the bottom, there is a text input field with the prompt "Ingresa el comando y presiona ENTER" and a green button labeled "crear aleatorio".

Hash

- Distribuye registros en cubetas usando función hash.
- Maneja colisiones con listas.
- Ventajas: búsqueda muy rápida ($O(1)$ promedio).
- Desventajas: complejidad en manejo de colisiones y tamaño de tabla.

The screenshot shows a window titled "sistema de archivo" with three tabs: "Algoritmos de planificación", "Algoritmos de reemplazo de pagina", and "sistema de archivo". The window is divided into three main sections:

- Left Panel (File Explorer):** Shows a directory structure with folders "uaemex" and "uvm", and files "alumno_56", "alumno_61", "alumno_73", "alumno_79", and "alumno_96".
- Center Panel (Terminal):** Displays a series of commands and their outputs in green text on a black background. The commands include creating random students, changing organization (pile, sec, hash), and listing files. The outputs show student names and their assigned hash values.
- Right Panel (Table):** A table with columns "Archivo", "Clave", and "Org Info". It lists the files and their corresponding hash values and organization types.

At the bottom of the window, there is a text input field with the prompt "Ingresa el comando y presiona ENTER" and a green button labeled "crear aleatorio".

Archivo	Clave	Org Info
alumno_7318....	7318	Hash bkt=30
alumno_7918....	7918	Hash bkt=73
alumno_6196....	6196	Hash bkt=88
alumno_5680....	5680	Hash bkt=91